

Assignment 4: Strategy and Implementation Hints

Approaching the Autoplayer Challenge

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Presentation Overview

Objectives

To provide hints on general strategy and approaches for writing the Tetris autoplayer. We will analyse the computational feasibility of different search strategies, discuss how to evaluate board positions, and review the critical data available within the codebase.

- 1 The Fundamental Assumption
- 2 Strategy 1: The Exhaustive Search
- 3 Strategy 2: Focusing on Target States
- 4 Evaluating the Board (Heuristics)
- 5 Implementation Details: Accessing Data
- 6 Summary

The Fundamental Assumption

The Fundamental Assumption

The Prerequisite: Scoring the Board

To develop an autoplayer, we must start with a fundamental assumption.

The Assumption

We assume we can develop a function that scores the quality of a board position immediately after a block lands and any rows are eliminated.



The Strategy Goal

If we have this scoring function, the goal of the AI (`choose_action`) becomes finding the sequence of moves that maximises this score when the block lands.

The Prerequisite: Scoring the Board

To develop an autoplayer, we must start with a fundamental assumption.

The Assumption

We assume we can develop a function that scores the quality of a board position immediately after a block lands and any rows are eliminated.



The Strategy Goal

If we have this scoring function, the goal of the AI (`choose_action`) becomes finding the sequence of moves that maximises this score when the block lands.

Strategy 1: The Exhaustive Search

Strategy 1: The Exhaustive Search

Idea 1: Trying All Possible Moves (Exhaustive Search)

A natural first approach is to explore every possible action at every step, similar to how we approached Noughts and Crosses (Week 5).

The Concept: Recursive Exploration (`try_all_moves`)

- 1 For all possible immediate moves:
- 2 Clone the board (essential!).
- 3 Make the move on the clone.
- 4 Did the block land?
 - **Yes:** Score the board (Base Case).
 - **No:** Recursively call `try_all_moves()` again (Recursive Step).
- 5 Keep track of the best score found.

Analysing the Exhaustive Search I

Will this strategy work?

Functionality

Yes, the algorithm is sound and will explore all possibilities until the block lands.

Computational Feasibility (The Problem)

We must analyse the complexity (Week 3).

- At any instant, there are 5 possible moves (Rotate CW/ACW, Left, Right, or do nothing/fall).
- The board has approximately 20 rows (the maximum depth of the search).

Analysing the Exhaustive Search I

Will this strategy work?

Functionality

Yes, the algorithm is sound and will explore all possibilities until the block lands.

Computational Feasibility (The Problem)

We must analyse the complexity (Week 3).

- At any instant, there are 5 possible moves (Rotate CW/ACW, Left, Right, or do nothing/fall).
- The board has approximately 20 rows (the maximum depth of the search).

Analysing the Exhaustive Search II

Will this strategy work?

The Complexity Calculation

The number of possibilities to explore for a single block landing is roughly:

$$5^{20}$$

The Conclusion

This approach is exponential and computationally intractable. The numbers are too big (5^{40} for two blocks). You cannot take the Noughts and Crosses approach; the AI will not complete figuring out what to do in time.

Analysing the Exhaustive Search II

Will this strategy work?

The Complexity Calculation

The number of possibilities to explore for a single block landing is roughly:

$$5^{20}$$

The Conclusion

This approach is exponential and computationally intractable. The numbers are too big (5^{40} for two blocks). You cannot take the Noughts and Crosses approach; the AI will not complete figuring out what to do in time.

Strategy 2: Focusing on Target States

Strategy 2: Focusing on Target States

Idea 2: Focusing on Landing Positions (Plan B)

If we cannot try all moves at each step, we must reduce the search space.

The Alternative Perspective

Instead of focusing on the sequence of moves, focus on all the possible ways the block might ultimately land.

Analyzing the Search Space

- There are approximately 10 columns where the block can land.
- There are at most 4 possible rotations for any block.

Idea 2: Focusing on Landing Positions (Plan B)

If we cannot try all moves at each step, we must reduce the search space.

The Alternative Perspective

Instead of focusing on the sequence of moves, focus on all the possible ways the block might ultimately land.

Analyzing the Search Space

- There are approximately 10 columns where the block can land.
- There are at most 4 possible rotations for any block.

Idea 2: Focusing on Landing Positions (Plan B)

The Alternative Perspective

Instead of focusing on the sequence of moves, focus on all the possible ways the block might ultimately land.

Analyzing the Search Space

- There are approximately 10 columns where the block can land.
- There are at most 4 possible rotations for any block.

The Reduced Complexity

Total possibilities $\approx 10 \text{ columns} \times 4 \text{ rotations} = 40 \text{ possibilities}$.

This is significantly better than 5^{20} and is computationally feasible.

The Target State Algorithm

This approach changes how we structure `choose_action()`.

The Algorithm

- 1 For all possible target Positions (Columns):
 - 2 For all possible target Rotations:
 - 1 Clone the board (Sandbox).
 - 2 Execute a sequence of moves to "Move Towards Target".
 - 3 Once the target is reached and the block lands, Score the Board.
- 3 Select the target (and the corresponding moves) that yields the best score.

The New Challenge

We now need an efficient implementation for "Move Towards Target".

The Target State Algorithm

This approach changes how we structure `choose_action()`.

The Algorithm

- 1 For all possible target Positions (Columns):
 - 2 For all possible target Rotations:
 - 1 Clone the board (Sandbox).
 - 2 Execute a sequence of moves to "Move Towards Target".
 - 3 Once the target is reached and the block lands, Score the Board.
- 3 Select the target (and the corresponding moves) that yields the best score.

The New Challenge

We now need an efficient implementation for "Move Towards Target".

Implementation Detail: Move Towards Target

How do we generate the sequence of moves to reach a specific target configuration?

The Task

Given a target position (X coordinate) and a target rotation, perform successive moves until the target is reached and the block lands.

The Strategic Question: Order of Operations

Should we rotate first, then move horizontally? Or move first, then rotate?

Implementation Detail: Move Towards Target

How do we generate the sequence of moves to reach a specific target configuration?

The Task

Given a target position (X coordinate) and a target rotation, perform successive moves until the target is reached and the block lands.

The Strategic Question: Order of Operations

Should we rotate first, then move horizontally? Or move first, then rotate?

Implementation Detail: Move Towards Target

The Task

Given a target position (X coordinate) and a target rotation, perform successive moves until the target is reached and the block lands.

The Strategic Question: Order of Operations

Should we rotate first, then move horizontally? Or move first, then rotate?

The Rotation Nuance

The order matters. When a block rotates (e.g., a long thin block), it might "push across" columns if it's near an obstruction. If you move first and then rotate, you may no longer be in the target column.

You must experiment to find most robust sequence (e.g., rotate first, move first, or alternate).

Evaluating the Board (Heuristics)

Evaluating the Board (Heuristics)

Designing the Score Function

The "Target State" strategy allows us to explore all 40 landing possibilities efficiently. Now we return to the initial assumption: How do we score the resulting board?

The Goal

Convert a board position into numerical score that reflects how "good" or "bad" a position is.

What Features Matter?

We need to identify features of the board that correlate with success or failure. Common factors include:

- **Height:** Is it worse if the blocks are higher up the board? (Yes).
- **Remaining Blocks:** Is it worse if there are more blocks on the board? (Yes, because the alternative is they have been eliminated).

Designing the Score Function

The "Target State" strategy allows us to explore all 40 landing possibilities efficiently. Now we return to the initial assumption: How do we score the resulting board?

The Goal

Convert a board position into numerical score that reflects how "good" or "bad" a position is.

What Features Matter?

We need to identify features of the board that correlate with success or failure. Common factors include:

- **Height:** Is it worse if the blocks are higher up the board? (Yes).
- **Remaining Blocks:** Is it worse if there are more blocks on the board? (Yes, because the alternative is they have been eliminated).

Designing the Score Function

The Goal

Convert a board position into numerical score that reflects how "good" or "bad" a position is.

What Features Matter?

We need to identify features of the board that correlate with success or failure. Common factors include:

- **Height:** Is it worse if the blocks are higher up the board? (Yes).
- **Remaining Blocks:** Is it worse if there are more blocks on the board? (Yes, because the alternative is they have been eliminated).
- **Holes:** Is it worse if there are "trapped holes" (empty spaces underneath landed blocks)? (Yes).

Designing the Score Function

What Features Matter?

We need to identify features of the board that correlate with success or failure. Common factors include:

- **Height:** Is it worse if the blocks are higher up the board? (Yes).
- **Remaining Blocks:** Is it worse if there are more blocks on the board? (Yes, because the alternative is they have been eliminated).
- **Holes:** Is it worse if there are "trapped holes" (empty spaces underneath landed blocks)? (Yes).

The Task

You must write an algorithm to quantify these features and combine them into a single score.

Implementation Details: Accessing Data

Implementation Details: Accessing Data

Available Information: The Board Object

To implement the scoring and the search, we must understand the data provided to `choose_action(board)`.

1. Landed Blocks: `board.cells`

- This is a Python `set` (behaves like a dictionary with no values).
- Contains the (x, y) coordinates of all blocks that have already landed.
- This defines the "landscape" of the board.

```
1 # Example Output (T-block at bottom):  
2 # {(5, 22), (6, 22), (7, 22), (6, 23)}
```

Available Information: Falling and Next Blocks

We also need information about the current and future blocks.

2. Falling Block: `board.falling.cells`

- A set of (x, y) coordinates for the block currently under the player's control.

3. Next Block: `board.next.cells`

- The coordinates of the next block that will appear.
- Useful for more advanced strategies (lookahead), but likely not needed for simpler solutions.

Data Structure Note

The data in these sets are not necessarily sorted into any particular order.

Available Information: Falling and Next Blocks

We also need information about the current and future blocks.

2. Falling Block: `board.falling.cells`

- A set of (x, y) coordinates for the block currently under the player's control.

3. Next Block: `board.next.cells`

- The coordinates of the next block that will appear.
- Useful for more advanced strategies (lookahead), but likely not needed for simpler solutions.

Data Structure Note

The data in these sets are not necessarily sorted into any particular order.

Available Information: Falling and Next Blocks

We also need information about the current and future blocks.

2. Falling Block: `board.falling.cells`

- A set of (x, y) coordinates for the block currently under the player's control.

3. Next Block: `board.next.cells`

- The coordinates of the next block that will appear.
- Useful for more advanced strategies (lookahead), but likely not needed for simpler solutions.

Data Structure Note

The data in these sets are not necessarily sorted into any particular order.

The Coordinate System

Understanding the coordinate system is crucial for implementing heuristics and movement logic.

The Grid

- **Origin (0,0):** Top-Left corner.
- **Rows (Y-axis):** Row 0 is the top; Row 23 (approximately) is the bottom.
- **Columns (X-axis):** Increase from left to right.

Practical Implementation Advice

Two final practical considerations for the implementation.

Handling Rotation

How do you determine the current rotation of the block when trying to reach the target rotation?

Advice: It is generally easier to keep track of how many times you have rotated the block yourself within your simulation, rather than trying to infer the rotation by looking at the cell positions.

Practical Implementation Advice

Handling Rotation

How do you determine the current rotation of the block when trying to reach the target rotation?

Advice: It is generally easier to keep track of how many times you have rotated the block yourself within your simulation, rather than trying to infer the rotation by looking at the cell positions.

Ensuring Landing

How do you ensure the block has landed so you can score the board in your simulation?

Advice: Once you have moved the block to the desired column and rotation, issue a `Direction.Drop` move. This forces the block to land immediately, after which you can confidently check the score.

Summary

Summary

Summary of Strategies I

The Algorithmic Choice

- **Idea 1 (Exhaustive DFS):** Trying all move sequences (like Minimax) is intractable ($\approx 5^{20}$ possibilities).
- **Idea 2 (Final States):** Focusing on the final landing positions (≈ 40 states) is efficient and feasible. This is the recommended approach.

Implementation Requirements

Summary of Strategies II

- 1 **Scoring Function:** Develop heuristics based on features like Height and Holes.
- 2 **Move Generation:** Implement a "Move Towards Target" function, carefully considering the order of rotation and translation.
- 3 **Cloning:** Essential for experimenting with moves (the sandbox).
- 4 **Data Access:** Understand `board.cells` and the coordinate system.